

Walchand College of Engineering, Sangli

Department of Computer Science & Engineering

Computer Algorithm Laboratory

Assignment No. 7 — Dynamic Programming

Name : Atharv V Kulkarni

PRN : 245109074

Class : TY B.Tech CSE (B)

Question 1: Hospital Surgery Scheduling with k Operating Rooms

Problem Statement:

A hospital has one operating room and receives n surgery requests for the day. Each surgery has a start time, an end time and a priority score reflecting medical urgency and hospital revenue, and only non-overlapping surgeries can be performed in the same room. Select a subset of non-overlapping surgeries that maximizes the total priority score. Then extend the solution to k operating rooms, where surgeries in different rooms can run in parallel but no two overlapping surgeries share a room. Report the maximum total priority and the schedule (which surgery goes to which room).

Approach:

approach_1: try every subset of surgeries and keep the best subset that never needs more than k rooms at the same time. tc: $O(2^n)$ sc: $O(n)$

approach_2 (optimal): for one room, sort by end time and use $dp[i] = \max(\text{skip surgery } i, \text{priority of } i + dp[\text{last compatible surgery}])$ with binary search. for k rooms, sort by start time and run a memoized dp over (surgery index, finish times of the busy rooms), at each surgery either skipping it or placing it in a free room, then assign rooms to the chosen surgeries. tc: $O(n \log n)$ for one room, $O(n^{k+1})$ for k rooms sc: $O(n)$ for one room, $O(n^{k+1})$ for k rooms

Time Complexity: $O(n \log n)$ for one room, $O(n^{k+1})$ for k rooms

Space Complexity: $O(n)$ for one room, $O(n^{k+1})$ for k rooms

Code :

```
#include<iostream>
#include<vector>
#include<algorithm>
#include<map>
using namespace std;
struct Surgery{int start,finish,priority,id;};
vector<Surgery>ops;
map<pair<int,vector<int>>,int>memo;
int solveRooms(int i,vector<int>busy){
    if(i==(int)ops.size())
        return 0;
    for(int&f:busy){
        if(f<=ops[i].start)
            f=0;
    }
    sort(busy.begin(),busy.end());
    pair<int,vector<int>>key=make_pair(i,busy);
    auto it=memo.find(key);
    if(it!=memo.end())
        return it->second;
    int best=solveRooms(i+1,busy);
    if(busy[0]==0){
        vector<int>nxt=busy;
        nxt[0]=ops[i].finish;
        best=max(best,ops[i].priority+solveRooms(i+1,nxt));
    }
    memo[key]=best;
    return best;
}
int multiRoom(vector<Surgery>all,int k,vector<pair<Surgery,int>>&plan){
    sort(all.begin(),all.end(),[](const Surgery&a,const Surgery&b){
        if(a.start!=b.start)
            return a.start<b.start;
        return a.finish<b.finish;
    });
    ops=all;
    memo.clear();
    int total=solveRooms(0,vector<int>(k,0));
    vector<int>busy(k,0);
    vector<Surgery>chosen;
    for(int i=0;i<(int)ops.size();i++){
        for(int&f:busy){
            if(f<=ops[i].start)
                f=0;
        }
        sort(busy.begin(),busy.end());
        int skip=solveRooms(i+1,busy);
```

```

    int take=-1;
    vector<int>nxt=busy;
    if(busy[0]==0){
        nxt[0]=ops[i].finish;
        take=ops[i].priority+solveRooms(i+1,nxt);
    }
    if(take>skip){
        chosen.push_back(ops[i]);
        busy=nxt;
    }
}
vector<int>roomFree(k,0);
for(Surgery&s:chosen){
    for(int r=0;r<k;r++){
        if(roomFree[r]<=s.start){
            roomFree[r]=s.finish;
            plan.push_back(make_pair(s,r+1));
            break;
        }
    }
}
return total;
}

int singleRoom(vector<Surgery>all,vector<Surgery>&chosen){
    sort(all.begin(),all.end(),[](const Surgery&a,const Surgery&b){
        return a.finish<b.finish;
    });
    int n=all.size();
    vector<int>dp(n+1,0),before(n+1,0);
    for(int i=1;i<=n;i++){
        int lo=0,hi=i-1;
        while(lo<hi){
            int mid=(lo+hi+1)/2;
            if(all[mid-1].finish<=all[i-1].start)
                lo=mid;
            else
                hi=mid-1;
        }
        before[i]=lo;
        dp[i]=max(dp[i-1],all[i-1].priority+dp[lo]);
    }
    int i=n;
    while(i>0){
        if(dp[i]==dp[i-1])
            i--;
        else{
            chosen.push_back(all[i-1]);
            i=before[i];
        }
    }
}

```

```

    }
}
reverse(chosen.begin(),chosen.end());
return dp[n];
}
vector<Surgery>build(vector<vector<int>>>data){
    vector<Surgery>all;
    for(int i=0;i<(int)data.size();i++){
        Surgery s;
        s.start=data[i][0];
        s.finish=data[i][1];
        s.priority=data[i][2];
        s.id=i+1;
        all.push_back(s);
    }
    return all;
}
void run(int tc,vector<Surgery>all,int k){
    vector<pair<Surgery,int>>plan;
    int best=multiRoom(all,k,plan);
    cout<<"test case "<<tc<<": rooms="<<k;
    cout<<" , maximum priority is "<<best<<endl;
    for(auto&p:plan){
        Surgery&s=p.first;
        cout<<" surgery "<<s.id<<" ("<<s.start<<"-<<s.finish;
        cout<<" , priority "<<s.priority<<" -> room "<<p.second<<endl;
    }
    if(k==1){
        vector<Surgery>chosen;
        cout<<" single room dp result is "<<singleRoom(all,chosen)<<endl;
    }
}
int main(){
    vector<Surgery>s1=build({{1,2,50},{3,5,20},{6,19,100},{2,100,200}});
    vector<Surgery>s2=build({{9,12,40},{10,14,50},{11,13,30},
        {12,15,60},{13,16,20}});
    vector<Surgery>s3=build({{1,4,10},{2,6,20},{3,5,15},
        {5,8,25},{7,10,30},{4,9,12}});
    run(1,s1,1);
    run(2,s1,2);
    run(3,s2,1);
    run(4,s2,3);
    run(5,s3,2);
    return 0;
}

```

Output:

```
*Linux@atharvk ~ main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_7 at 02:44:56 PM
➤ mkdir -p build && g++ q1_surgery_scheduling.cpp -o build/q1_surgery_scheduling && ./build/q1_surgery_scheduling
test case 1: rooms=1, maximum priority is 250
  surgery 1 (1-2, priority 50) -> room 1
  surgery 4 (2-100, priority 200) -> room 1
  single room dp result is 250
test case 2: rooms=2, maximum priority is 370
  surgery 1 (1-2, priority 50) -> room 1
  surgery 4 (2-100, priority 200) -> room 1
  surgery 2 (3-5, priority 20) -> room 2
  surgery 3 (6-19, priority 100) -> room 2
test case 3: rooms=1, maximum priority is 100
  surgery 1 (9-12, priority 40) -> room 1
  surgery 4 (12-15, priority 60) -> room 1
  single room dp result is 100
test case 4: rooms=3, maximum priority is 200
  surgery 1 (9-12, priority 40) -> room 1
  surgery 2 (10-14, priority 50) -> room 2
  surgery 3 (11-13, priority 30) -> room 3
  surgery 4 (12-15, priority 60) -> room 1
  surgery 5 (13-16, priority 20) -> room 3
test case 5: rooms=2, maximum priority is 90
  surgery 2 (2-6, priority 20) -> room 1
  surgery 3 (3-5, priority 15) -> room 2
  surgery 4 (5-8, priority 25) -> room 2
  surgery 5 (7-10, priority 30) -> room 1
```

Question 2: Factory Job Sequencing with Late Penalties

Problem Statement:

A factory has n production jobs and a single production line that runs one job at a time without preemption. Each job has a processing time, a deadline and a penalty that is paid if the job finishes after its deadline (the job still gets completed, just late). Find the job order that minimizes the total penalty incurred across all jobs and report the minimum penalty along with the optimal order.

Approach:

approach_1: try every possible job order and take the order with the smallest total penalty. tc: $O(n \text{ into } n!)$ sc: $O(n)$

approach_2 (optimal): sort jobs by deadline and use $dp[i][t]$ = minimum penalty after deciding the first i jobs when the on-time jobs use total time t . each job is either kept on time (if $t + \text{its time} \leq \text{its deadline}$) or pushed to the end as a late job that pays its penalty. final order is the on-time jobs by deadline followed by the late jobs. tc: $O(n \text{ into total processing time})$ sc: $O(n \text{ into total processing time})$

Time Complexity: $O(n \text{ into total processing time})$

Space Complexity: $O(n \text{ into total processing time})$

Code :

```
#include<iostream>
#include<vector>
#include<algorithm>
#include<climits>
using namespace std;
struct Job{int time,deadline,penalty,id;};
int minPenalty(vector<Job>jobs,vector<int>&order){
    sort(jobs.begin(),jobs.end(),[](const Job&a,const Job&b){
        return a.deadline<b.deadline;
    });
    int n=jobs.size(),total=0;
    for(Job&j;jobs)
        total+=j.time;
    const int INF=INT_MAX/2;
    vector<vector<int>>>dp(n+1,vector<int>(total+1,INF));
    vector<vector<int>>>onTime(n+1,vector<int>(total+1,0));
    dp[0][0]=0;
    for(int i=1;i<=n;i++){
        Job&j=jobs[i-1];
        for(int t=0;t<=total;t++){
            if(dp[i-1][t]>=INF)
                continue;
            if(dp[i-1][t]+j.penalty<dp[i][t]){
                dp[i][t]=dp[i-1][t]+j.penalty;
                onTime[i][t]=0;
            }
        }
    }
}
```

```

        if(t+j.time<=j.deadline&&dp[i-1][t]<dp[i][t+j.time]){
            dp[i][t+j.time]=dp[i-1][t];
            onTime[i][t+j.time]=1;
        }
    }
}
int bestT=0;
for(int t=1;t<=total;t++){
    if(dp[n][t]<dp[n][bestT])
        bestT=t;
}
int result=dp[n][bestT];
vector<int>early,late;
int t=bestT;
for(int i=n;i>=1;i--){
    if(onTime[i][t]){
        early.push_back(jobs[i-1].id);
        t-=jobs[i-1].time;
    }
    else
        late.push_back(jobs[i-1].id);
}
reverse(early.begin(),early.end());
reverse(late.begin(),late.end());
order=early;
order.insert(order.end(),late.begin(),late.end());
return result;
}
void run(int tc,vector<vector<int>>data){
    vector<Job>jobs;
    for(int i=0;i<(int)data.size();i++){
        Job j;
        j.time=data[i][0];
        j.deadline=data[i][1];
        j.penalty=data[i][2];
        j.id=i+1;
        jobs.push_back(j);
    }
    vector<int>order;
    int best=minPenalty(jobs,order);
    cout<<"test case "<<tc<<": minimum penalty is "<<best<<endl;
    cout<<" job order:";
    for(int id:order)
        cout<<" J"<<id;
    cout<<endl;
    int clock=0;
    for(int id:order){
        Job&j=jobs[id-1];

```

```

        clock+=j.time;
        cout<<" J"<<id<<" finishes at "<<clock;
        cout<<" (deadline "<<j.deadline<<" )";
        if(clock>j.deadline)
            cout<<"late, penalty "<<j.penalty<<endl;
        else
            cout<<"on time"<<endl;
    }
}
int main(){
    run(1,{{2,3,10},{1,2,20},{3,7,15},{2,6,5},{4,9,25}});
    run(2,{{4,4,10},{3,5,20},{2,6,30}});
    run(3,{{3,3,5},{3,3,8},{3,3,6}});
    run(4,{{5,20,40},{2,4,10},{3,8,25},{4,9,30},{1,3,15},{2,10,12}});
    run(5,{{6,5,50}});
    return 0;
}

```

Output:

```

Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_7 at 02:46:04 PM
➤ mkdir -p build && g++ q2_job_sequencing.cpp -o build/q2_job_sequencing && ./build/q2_job_sequencing
test case 1: minimum penalty is 15
job order: J2 J3 J5 J1 J4
J2 finishes at 1 (deadline 2) on time
J3 finishes at 4 (deadline 7) on time
J5 finishes at 8 (deadline 9) on time
J1 finishes at 10 (deadline 3) late, penalty 10
J4 finishes at 12 (deadline 6) late, penalty 5
test case 2: minimum penalty is 10
job order: J2 J3 J1
J2 finishes at 3 (deadline 5) on time
J3 finishes at 5 (deadline 6) on time
J1 finishes at 9 (deadline 4) late, penalty 10
test case 3: minimum penalty is 11
job order: J2 J1 J3
J2 finishes at 3 (deadline 3) on time
J1 finishes at 6 (deadline 3) late, penalty 5
J3 finishes at 9 (deadline 3) late, penalty 6
test case 4: minimum penalty is 10
job order: J5 J3 J4 J6 J1 J2
J5 finishes at 1 (deadline 3) on time
J3 finishes at 4 (deadline 8) on time
J4 finishes at 8 (deadline 9) on time
J6 finishes at 10 (deadline 10) on time
J1 finishes at 15 (deadline 20) on time
J2 finishes at 17 (deadline 4) late, penalty 10
test case 5: minimum penalty is 50
job order: J1
J1 finishes at 6 (deadline 5) late, penalty 50

```


Question 3: University Exam Scheduling

Problem Statement:

A university must schedule n exams into time slots. Some pairs of exams conflict because they share students and cannot be placed in the same slot, and each exam has a difficulty weight. Part A: model the conflicts as a graph and use graph coloring to find the minimum number of slots needed. Part B: given the minimum slot count, use bitmask dp over subsets to also minimize the penalty for conflicting high-difficulty exams placed in adjacent slots (an exam is high difficulty when its weight is 7 or more, and the penalty is the sum of the two weights). Report the minimum number of slots, the exam-to-slot assignment and the total penalty.

Approach:

approach_1: try every assignment of exams to k slots for $k = 1$ to n , reject the ones with a conflict inside a slot, and keep the smallest k with the least penalty. tc: $O(n^4)$ sc: $O(n)$

approach_2 (optimal): part a uses bitmask dp where $dp[mask] =$ minimum colors for the exams in mask, choosing an independent subset that contains the lowest exam of mask. part b uses $dp[j][mask][last] =$ minimum penalty after filling j slots with the exams in mask where last is the set placed in the latest slot, adding the adjacent-slot penalty each time the next independent set is chosen. tc: $O(k^4 \cdot n^2)$ sc: $O(k^4 \cdot n)$

Time Complexity: $O(k^4 \cdot n^2)$

Space Complexity: $O(k^4 \cdot n)$

Code :

```
#include<iostream>
#include<vector>
#include<utility>
using namespace std;
const int INF=1000000000;
const int HIGH=7;
int n;
vector<int>adj,weight;
vector<char>indep;
void setup(int count,vector<pair<int,int>>conflicts,vector<int>w){
    n=count;
    weight=w;
    adj.assign(n,0);
    for(auto&c:conflicts){
        adj[c.first]|=1<<c.second;
        adj[c.second]|=1<<c.first;
    }
    indep.assign(1<<n,1);
    for(int mask=0;mask<(1<<n);mask++){
        for(int i=0;i<n;i++){
```

```

        if(((mask>>i)&1)&&(adj[i]&mask))
            indep[mask]=0;
    }
}
}
int minSlots(){
    int full=(1<<n)-1;
    vector<int>dp(full+1,INF);
    dp[0]=0;
    for(int mask=1;mask<=full;mask++){
        int low=mask&-mask;
        for(int sub=mask;sub>0;sub=(sub-1)&mask){
            if((sub&low)&&indep[sub]&&dp[mask^sub]+1<dp[mask])
                dp[mask]=dp[mask^sub]+1;
        }
    }
    return dp[full];
}
int pairCost(int a,int b){
    int cost=0;
    for(int i=0;i<n;i++){
        if(!((a>>i)&1))
            continue;
        for(int j=0;j<n;j++){
            if(((b>>j)&1)&&((adj[i]>>j)&1)&&weight[i]>=HIGH&&weight[j]>=HIGH)
                cost+=weight[i]+weight[j];
        }
    }
    return cost;
}
int arrange(int slots,vector<int>&slotOf){
    int full=(1<<n)-1;
    vector<vector<int>>>table(full+1,vector<int>(full+1,INF));
    vector<vector<vector<int>>>dp(slots+1,table);
    table.assign(full+1,vector<int>(full+1,-1));
    vector<vector<vector<int>>>par(slots+1,table);
    for(int s=1;s<=full;s++){
        if(indep[s])
            dp[1][s][s]=0;
    }
    for(int j=1;j<slots;j++){
        for(int mask=1;mask<=full;mask++){
            for(int last=mask;last>0;last=(last-1)&mask){
                int cur=dp[j][mask][last];
                if(cur>=INF)
                    continue;
                int rest=full^mask;
                for(int nxt=rest;nxt>0;nxt=(nxt-1)&rest){

```

```

        if(!indep[nxt])
            continue;
        int c=cur+pairCost(last,nxt);
        if(c<dp[j+1][mask|nxt][nxt]){
            dp[j+1][mask|nxt][nxt]=c;
            par[j+1][mask|nxt][nxt]=last;
        }
    }
}
}
}
int best=INF,bestLast=0;
for(int last=1;last<=full;last++){
    if(dp[slots][full][last]<best){
        best=dp[slots][full][last];
        bestLast=last;
    }
}
int mask=full,last=bestLast;
slotOf.assign(n,0);
for(int j=slots;j>=1;j--){
    for(int i=0;i<n;i++){
        if((last>>i)&1)
            slotOf[i]=j;
    }
    int before=par[j][mask][last];
    mask^=last;
    last=before;
}
return best;
}
void run(int tc,int count,vector<pair<int,int>>conflicts,vector<int>w){
    setup(count,conflicts,w);
    int k=minSlots();
    vector<int>slotOf;
    int penalty=arrange(k,slotOf);
    cout<<"test case "<<tc<<": minimum number of slots is "<<k<<endl;
    for(int s=1;s<=k;s++){
        cout<<" slot "<<s<<". ";
        for(int i=0;i<n;i++){
            if(slotOf[i]==s)
                cout<<" E"<<i+1<<"(d="<<weight[i]<<")";
        }
        cout<<endl;
    }
    cout<<" total adjacent difficulty penalty is "<<penalty<<endl;
}
int main(){

```

```

run(1,5,{{0,1},{0,2},{1,2},{2,3},{3,4}},{8,9,7,3,8});
run(2,6,{{0,1},{1,2},{2,3},{3,4},{4,5},{5,0}},{9,4,8,3,9,8});
run(3,4,{{0,1},{0,2},{0,3},{1,2},{1,3},{2,3}},{9,2,8,7});
run(4,7,{{5,9,3,8,7,2,10}});
run(5,8,{{0,1},{1,2},{2,3},{3,4},{4,0},{5,6},{6,7},{5,7},{0,5},{2,6}},
    {8,7,9,3,10,8,2,9});
return 0;
}

```

Output:

```

Linux@atharvk % main in ~/Documents/WLE_Lab_Material/1Y_Computer_Algorithm_Lab/2451090/4_LAL_Assignment_7 at 02:46:38 PM
➤ mkdir -p build && g++ q3_exam_scheduling.cpp -o build/q3_exam_scheduling && ./build/q3_exam_scheduling
test case 1: minimum number of slots is 3
slot 1: E2(d=9) E4(d=3)
slot 2: E3(d=7) E5(d=8)
slot 3: E1(d=8)
total adjacent difficulty penalty is 31
test case 2: minimum number of slots is 2
slot 1: E2(d=4) E4(d=3) E6(d=8)
slot 2: E1(d=9) E3(d=8) E5(d=9)
total adjacent difficulty penalty is 34
test case 3: minimum number of slots is 4
slot 1: E3(d=8)
slot 2: E4(d=7)
slot 3: E2(d=2)
slot 4: E1(d=9)
total adjacent difficulty penalty is 15
test case 4: minimum number of slots is 1
slot 1: E1(d=5) E2(d=9) E3(d=3) E4(d=8) E5(d=7) E6(d=2) E7(d=10)
total adjacent difficulty penalty is 0
test case 5: minimum number of slots is 3
slot 1: E1(d=8) E3(d=9) E8(d=9)
slot 2: E4(d=3) E7(d=2)
slot 3: E2(d=7) E5(d=10) E6(d=8)
total adjacent difficulty penalty is 0

```

Question 4: Optimal Binary Search Tree for a Spell Checker

Problem Statement:

A spell-checker looks up n keywords, each with a known search frequency. Build a binary search tree over the sorted keywords so that the average search cost (number of comparisons) is minimized, so that frequently searched words sit closer to the root. Given n keys with frequencies $p_1 \dots p_n$ (and optionally dummy-key probabilities $q_0 \dots q_n$ for unsuccessful searches), construct the optimal binary search tree, report its minimum expected search cost and show the root chosen for each subrange.

Approach:

approach_1: recursively try every key as the root of every range and take the cheapest tree. tc: $O(4^n)$ sc: $O(n)$

approach_2 (optimal): bottom-up dp where $e[i][j]$ is the expected cost of the best tree on keys i to j , $w[i][j]$ is the total probability of that range, and $root[i][j]$ stores the best root, trying every key r in the range as the root. tc: $O(n^3)$ sc: $O(n^2)$

Time Complexity: $O(n^3)$

Space Complexity: $O(n^2)$

Code :

```
#include<iostream>
#include<vector>
#include<string>
#include<iomanip>
using namespace std;
double optimalBST(int n,vector<double>&p,vector<double>&q,
vector<vector<int>>&root){
vector<vector<double>>e(n+2,vector<double>(n+1,0));
vector<vector<double>>w(n+2,vector<double>(n+1,0));
root.assign(n+2,vector<int>(n+1,0));
for(int i=1;i<=n+1;i++){
e[i][i-1]=q[i-1];
w[i][i-1]=q[i-1];
}
for(int len=1;len<=n;len++){
for(int i=1;i<=n-len+1;i++){
int j=i+len-1;
e[i][j]=1e18;
w[i][j]=w[i][j-1]+p[j]+q[j];
for(int r=i;r<=j;r++){
double t=e[i][r-1]+e[r+1][j]+w[i][j];
if(t<e[i][j]){
e[i][j]=t;
root[i][j]=r;
}
}
}
}
```

```

    }
    return e[1][n];
}

void showTree(vector<vector<int>>&root,vector<string>&keys,int i,int j,
    int parent,string side){
    if(i>j)
        return;
    int r=root[i][j];
    if(parent==0){
        cout<<" "<<keys[r]<<" is the root of keys ";
        cout<<keys[i]<<" to "<<keys[j]<<endl;
    }
    else{
        cout<<" "<<keys[r]<<" is the "<<side<<" child of "<<keys[parent];
        cout<<" (keys "<<keys[i]<<" to "<<keys[j]<<")"<<endl;
    }
    showTree(root,keys,i,r-1,r,"left");
    showTree(root,keys,r+1,j,r,"right");
}

void run(int tc,vector<string>words,vector<double>p,vector<double>q){
    int n=words.size();
    vector<string>keys(1,"");
    for(string&s:words)
        keys.push_back(s);
    p.insert(p.begin(),0.0);
    vector<vector<int>>root;
    double cost=optimalBST(n,p,q,root);
    cout<<"test case "<<tc<<": minimum expected search cost is "<<cost<<endl;
    cout<<" root of every subrange:"<<endl;
    for(int i=1;i<=n;i++){
        for(int j=i;j<=n;j++){
            cout<<" keys "<<keys[i]<<" to "<<keys[j];
            cout<<" -> root "<<keys[root[i][j]]<<endl;
        }
    }
    cout<<" tree structure:"<<endl;
    showTree(root,keys,1,n,0,"");
}

int main(){
    cout<<fixed<<setprecision(4);
    run(1,{"array","binary","cache","debug","event"},
        {0.15,0.10,0.05,0.10,0.20},{0.05,0.10,0.05,0.05,0.10});
    run(2,{"cat","dog","fox"},{0.5,0.3,0.2},{0,0,0});
    run(3,{"ant","bee","cow"},{34,8,50},{0,0,0});
    run(4,{"spell"},{1.0},{0,0});
    run(5,{"check","edit","find","open"},{0.25,0.15,0.20,0.10},
        {0.06,0.06,0.06,0.06,0.06});
    return 0;
}

```

```
}
```

Output:

```
Linux@atharvk ~$ main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_7 at 02:47:05 PM
> mkdir -p build && g++ q4_optimal_bst.cpp -o build/q4_optimal_bst && ./build/q4_optimal_bst
tree structure:
cat is the root of keys cat to fox
dog is the right child of cat (keys dog to fox)
fox is the right child of dog (keys fox to fox)
test case 3: minimum expected search cost is 142.0000
root of every subrange:
keys ant to ant -> root ant
keys ant to bee -> root ant
keys ant to cow -> root cow
keys bee to bee -> root bee
keys bee to cow -> root cow
keys cow to cow -> root cow
tree structure:
cow is the root of keys ant to cow
ant is the left child of cow (keys ant to bee)
bee is the right child of ant (keys bee to bee)
test case 4: minimum expected search cost is 1.0000
root of every subrange:
keys spell to spell -> root spell
tree structure:
spell is the root of keys spell to spell
test case 5: minimum expected search cost is 2.3700
root of every subrange:
keys check to check -> root check
keys check to edit -> root check
keys check to find -> root edit
keys check to open -> root edit
keys edit to edit -> root edit
keys edit to find -> root find
keys edit to open -> root find
keys find to find -> root find
keys find to open -> root find
keys open to open -> root open
tree structure:
edit is the root of keys check to open
check is the left child of edit (keys check to check)
find is the right child of edit (keys find to open)
open is the right child of find (keys open to open)
```

Question 5: Fiber-Optic Network Using Prim's Algorithm

Problem Statement:

An ISP wants to connect n rural towns with fiber-optic cable so that every town is connected, directly or indirectly, at minimum total cost, and it wants to build the network incrementally starting from its main hub town. The cost of laying cable between towns is known. Using Prim's Algorithm find the minimum spanning tree starting from the hub town and report the total cost and the cables to be laid.

Approach:

approach_1: try every set of $n-1$ cables, keep the sets that connect all towns and take the cheapest. tc: $O(2^n)$ raised to e sc: $O(e)$

approach_2 (optimal): prim's greedy algorithm that starts at the hub and repeatedly adds the cheapest cable joining a town outside the network to a town already inside it, using key and parent arrays over an adjacency matrix. tc: $O(n^2)$ sc: $O(n^2)$

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$

Code :

```
#include<iostream>
#include<vector>
#include<climits>
using namespace std;
struct Edge{int u,v,cost;};
void primMST(int tc,int n,vector<Edge>edges,int hub){
    vector<vector<int>>>g(n,vector<int>(n,0));
    for(Edge&e:edges){
        g[e.u][e.v]=e.cost;
        g[e.v][e.u]=e.cost;
    }
    vector<int>key(n,INT_MAX),parent(n,-1);
    vector<bool>used(n,false);
    vector<Edge>cables;
    key[hub]=0;
    int total=0,reached=0;
    for(int step=0;step<n;step++){
        int u=-1;
        for(int v=0;v<n;v++){
            if(!used[v]&&(u==-1 | key[v]<key[u]))
                u=v;
        }
        if(key[u]==INT_MAX)
            break;
        used[u]=true;
        reached++;
        if(parent[u]!=-1){
            total+=key[u];
        }
    }
}
```



```

        cables.push_back({parent[u],u,key[u]});
    }
    for(int v=0;v<n;v++){
        if(g[u][v]>0&&!used[v]&&g[u][v]<key[v]){
            key[v]=g[u][v];
            parent[v]=u;
        }
    }
}
cout<<"test case "<<tc<<": hub town is "<<hub<<endl;
if(reached<n){
    cout<<" network cannot reach all towns, only "<<reached;
    cout<<" of "<<n<<" towns connected"<<endl;
    return;
}
cout<<" total minimum cost is "<<total<<endl;
for(Edge&c:cables){
    cout<<" cable: town "<<c.u<<" - town "<<c.v;
    cout<<" (cost "<<c.cost<<)"<<endl;
}
}
int main(){
    primMST(1,5,{{0,1,2},{0,3,6},{1,2,3},{1,3,8},{1,4,5},{2,4,7},{3,4,9}},0);
    primMST(2,4,{{0,1,10},{0,2,6},{0,3,5},{1,3,15},{2,3,4}},2);
    primMST(3,6,{{0,1,4},{0,2,4},{1,2,2},{2,3,3},{2,5,2},{2,4,4},{3,4,3},
        {5,4,3}},3);
    primMST(4,2,{{0,1,7}},1);
    primMST(5,4,{{0,1,3},{2,3,5}},0);
    return 0;
}

```

Output:

```
*Linux@atharvk % main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_7 at 02:47:43 PM
➤ mkdir -p build && g++ q5_prim_fiber_network.cpp -o build/q5_prim_fiber_network && ./build/q5_prim_fiber_network
test case 1: hub town is 0
total minimum cost is 16
cable: town 0 - town 1 (cost 2)
cable: town 1 - town 2 (cost 3)
cable: town 1 - town 4 (cost 5)
cable: town 0 - town 3 (cost 6)
test case 2: hub town is 2
total minimum cost is 19
cable: town 2 - town 3 (cost 4)
cable: town 3 - town 0 (cost 5)
cable: town 0 - town 1 (cost 10)
test case 3: hub town is 3
total minimum cost is 14
cable: town 3 - town 2 (cost 3)
cable: town 2 - town 1 (cost 2)
cable: town 2 - town 5 (cost 2)
cable: town 3 - town 4 (cost 3)
cable: town 2 - town 0 (cost 4)
test case 4: hub town is 1
total minimum cost is 7
cable: town 1 - town 0 (cost 7)
test case 5: hub town is 0
network cannot reach all towns, only 2 of 4 towns connected
```

Question 6: Power Grid Connection Using Kruskal's Algorithm

Problem Statement:

A national power company must connect n cities to the power grid at minimum total transmission-line cost. It has no fixed starting hub, so it evaluates all possible connections, and some city pairs cannot be connected directly (missing edges). Using Kruskal's Algorithm with Union-Find find the minimum spanning tree, and detect and report if the graph is disconnected along with its disconnected components. Report the total minimum cost, the selected edges and any disconnected components.

Approach:

approach_1: try every subset of edges of size $n-1$, keep the ones that connect all cities without a cycle and take the cheapest. tc: $O(2^n)$ sc: $O(n)$

approach_2 (optimal): sort all edges by cost and use union-find with path compression and union by rank to add an edge only when it joins two different components. after all edges are processed, group the cities by their root to report the components when the graph is disconnected. tc: $O(n \log n)$ sc: $O(n)$

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

Code :

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
struct Edge{int u,v,cost;};
vector<int>parent,rnk;
int findSet(int x){
    if(parent[x]!=x)
        parent[x]=findSet(parent[x]);
    return parent[x];
}
bool unite(int a,int b){
    a=findSet(a);
    b=findSet(b);
    if(a==b)
        return false;
    if(rnk[a]<rnk[b])
        swap(a,b);
    parent[b]=a;
    if(rnk[a]==rnk[b])
        rnk[a]++;
    return true;
}
void kruskal(int tc,int n,vector<Edge>edges){
```

```

parent.assign(n,0);
rnk.assign(n,0);
for(int i=0;i<n;i++)
    parent[i]=i;
sort(edges.begin(),edges.end(),[](const Edge&a,const Edge&b){
    return a.cost<b.cost;
});
int total=0;
vector<Edge>chosen;
for(Edge&e:edges){
    if(unite(e.u,e.v)){
        total+=e.cost;
        chosen.push_back(e);
    }
}
vector<vector<int>>>comp(n);
for(int i=0;i<n;i++)
    comp[findSet(i)].push_back(i);
int parts=0;
for(int i=0;i<n;i++){
    if(!comp[i].empty())
        parts++;
}
cout<<"test case "<<tc<<": ";
if(parts==1)
    cout<<"graph is connected, total minimum cost is "<<total<<endl;
else{
    cout<<"graph is disconnected with "<<parts<<" components, ";
    cout<<"minimum spanning forest cost is "<<total<<endl;
}
for(Edge&e:chosen){
    cout<<" selected edge: city "<<e.u<<" - city "<<e.v;
    cout<<" (cost "<<e.cost<<)"<<endl;
}
if(parts>1){
    int id=1;
    for(int i=0;i<n;i++){
        if(comp[i].empty())
            continue;
        cout<<" component "<<id++<<": ";
        for(int city:comp[i])
            cout<<" "<<city;
        cout<<endl;
    }
}
}
int main(){
    kruskal(1,6,{0,1,4},{0,2,4},{1,2,2},{2,3,3},{2,5,2},{2,4,4},{3,4,3},

```

```

        {5,4,3}));
kruskal(2,6,{{0,1,3},{1,2,1},{0,2,4},{3,4,2},{4,5,5},{3,5,6}});
kruskal(3,5,{{0,1,7},{1,2,2},{0,2,9}});
kruskal(4,4,{});
kruskal(5,5,{{0,1,1},{1,2,2},{2,3,3},{3,4,4},{4,0,5},{0,2,6},{1,3,7},
        {2,4,8}});
return 0;
}

```

Output:

```

Linux@atharvk % main in ~/Documents/WCE Lab Material/TY Computer Algorithm Lab/245109074 CAL Assignment 7 at 02:48:17 PM
• mkdir -p build && g++ q6_kruskal_power_grid.cpp -o build/q6_kruskal_power_grid && ./build/q6_kruskal_power_grid
test case 1: graph is connected, total minimum cost is 14
selected edge: city 1 - city 2 (cost 2)
selected edge: city 2 - city 5 (cost 2)
selected edge: city 2 - city 3 (cost 3)
selected edge: city 3 - city 4 (cost 3)
selected edge: city 0 - city 1 (cost 4)
test case 2: graph is disconnected with 2 components, minimum spanning forest cost is 11
selected edge: city 1 - city 2 (cost 1)
selected edge: city 3 - city 4 (cost 2)
selected edge: city 0 - city 1 (cost 3)
selected edge: city 4 - city 5 (cost 5)
component 1: 0 1 2
component 2: 3 4 5
test case 3: graph is disconnected with 3 components, minimum spanning forest cost is 9
selected edge: city 1 - city 2 (cost 2)
selected edge: city 0 - city 1 (cost 7)
component 1: 0 1 2
component 2: 3
component 3: 4
test case 4: graph is disconnected with 4 components, minimum spanning forest cost is 0
component 1: 0
component 2: 1
component 3: 2
component 4: 3
test case 5: graph is connected, total minimum cost is 10
selected edge: city 0 - city 1 (cost 1)
selected edge: city 1 - city 2 (cost 2)
selected edge: city 2 - city 3 (cost 3)
selected edge: city 3 - city 4 (cost 4)

```

Question 7: Drone Delivery Route Using Held-Karp (TSP)

Problem Statement:

A delivery drone must visit n delivery points exactly once each and return to the base warehouse while minimizing the total distance travelled. The distance between every pair of points is known and n is small (up to 15-18). Solve the Traveling Salesman Problem exactly using bitmask dynamic programming (Held-Karp algorithm) and report the minimum total route distance and the optimal visiting order.

Approach:

approach_1: try every possible visiting order of the points and keep the shortest round trip. tc: $O(n!)$ sc: $O(n)$

approach_2 (optimal): held-karp dp where $dp[mask][last]$ is the shortest path from the base that visits exactly the points in mask and ends at last, extended one unvisited point at a time, then closed by returning to the base. a parent table is used to rebuild the route. tc: $O(2^n \cdot n^2)$ sc: $O(2^n \cdot n)$

Time Complexity: $O(2^n \cdot n^2)$

Space Complexity: $O(2^n \cdot n)$

Code :

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
const int INF=1000000000;
int heldKarp(vector<vector<int>>&dist,int base,vector<int>&route){
    int n=dist.size();
    route.clear();
    if(n==1){
        route.push_back(base);
        route.push_back(base);
        return 0;
    }
    int full=(1<<n)-1;
    vector<vector<int>>dp(1<<n,vector<int>(n,INF));
    vector<vector<int>>par(1<<n,vector<int>(n,-1));
    dp[1<<base][base]=0;
    for(int mask=0;mask<=full;mask++){
        if(!((mask>>base)&1))
            continue;
        for(int last=0;last<n;last++){
            if(!((mask>>last)&1) || dp[mask][last]>=INF)
                continue;
            for(int nxt=0;nxt<n;nxt++){
                if((mask>>nxt)&1)
                    continue;
                int nm=mask|(1<<nxt);
```

```

        int c=dp[mask][last]+dist[last][nxt];
        if(c<dp[nm][nxt]){
            dp[nm][nxt]=c;
            par[nm][nxt]=last;
        }
    }
}
}
int best=INF,lastCity=-1;
for(int last=0;last<n;last++){
    if(last==base || dp[full][last]>=INF)
        continue;
    if(dp[full][last]+dist[last][base]<best){
        best=dp[full][last]+dist[last][base];
        lastCity=last;
    }
}
int mask=full,cur=lastCity;
while(cur!=-1){
    route.push_back(cur);
    int before=par[mask][cur];
    mask^=1<<cur;
    cur=before;
}
reverse(route.begin(),route.end());
route.push_back(base);
return best;
}
void run(int tc,vector<vector<int>>dist,int base){
    vector<int>route;
    int best=heldKarp(dist,base,route);
    cout<<"test case "<<tc<<": base warehouse is "<<base;
    cout<<", minimum route distance is "<<best<<endl;
    cout<<" route:";
    for(int i=0;i<(int)route.size();i++){
        if(i>0)
            cout<<" ->";
        cout<<" "<<route[i];
    }
    cout<<endl;
}
int main(){
    vector<vector<int>>d1={{0,10,15,20},{10,0,35,25},{15,35,0,30},
        {20,25,30,0}};
    vector<vector<int>>d2={{0,2,9,10,7},{1,0,6,4,3},{15,7,0,8,3},
        {6,3,12,0,11},{9,7,5,6,0}};
    vector<vector<int>>d3={{0,3,4,2,7,5},{3,0,4,6,3,8},{4,4,0,5,8,2},
        {2,6,5,0,6,4},{7,3,8,6,0,5},{5,8,2,4,5,0}};

```

```
run(1,d1,0);
run(2,d1,2);
run(3,d2,0);
run(4,{{0,12},{12,0}},1);
run(5,d3,3);
return 0;
}
```

Output:

```
Linux@atharvk % main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_7 at 02:48:43 PM
➤ mkdir -p build && g++ q7_tsp_held_karp.cpp -o build/q7_tsp_held_karp && ./build/q7_tsp_held_karp
test case 1: base warehouse is 0, minimum route distance is 80
route: 0 -> 2 -> 3 -> 1 -> 0
test case 2: base warehouse is 2, minimum route distance is 80
route: 2 -> 3 -> 1 -> 0 -> 2
test case 3: base warehouse is 0, minimum route distance is 22
route: 0 -> 2 -> 4 -> 3 -> 1 -> 0
test case 4: base warehouse is 1, minimum route distance is 24
route: 1 -> 0 -> 1
test case 5: base warehouse is 3, minimum route distance is 20
route: 3 -> 2 -> 5 -> 4 -> 1 -> 0 -> 3
```